Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Hata bulma ve düzeltme (slayt başlığı “Hata Bulma ve Düzeltme-2”, ekrandan, 00:10:00)
  - Hata, veri bir noktadan diğerine iletilirken bitlerde oluşan bozulmadır. Ortama bağlıdır: yıldırım, şimşek gibi atmosferik olaylar telli ya da telsiz ortamda hataya yol açabilir.
  - Gönderilen verinin arkasına kontrol bitleri eklenir; alıcı bu bitleri kontrol ederek hata olup olmadığını anlar. Hata varsa mesaj kabul edilmez ya da hata bulunup düzeltilebiliyorsa düzeltilir.
  - Tek bitlik bozulma bile verinin yanlış anlaşılmasına yol açabilir; bir bit 0’dan 1’e ya da 1’den 0’a döndüğünde verinin içeriği değişir.
  - Hatayı sezmek, düzeltmekten kolaydır. Çoğu zaman hata olduğu anlaşılır ama düzeltilemez. Düzeltmek, veriyi yeniden göndermekten daha maliyetli olduğu için yeniden gönderim istenir.
  - Tek bitlik bozulmaları sezmek ve düzeltmek kolaydır. Örneğin 12 bitlik bir blokta 2, 3 ya da 4 bit bozulmuşsa düzeltmek zordur, hatta imkânsızdır; ama bozulma olduğu anlaşılıp karşı tarafa bildirilebilir.
- Hata türleri
  - Burst error (patlama hatası): çevre koşulları nedeniyle alıcıya gerçek olmayan, anlamsız bilgiler gelir; bilginin birçok yeri bozulur.
  - Random error (rastgele hata): iletim yolundaki elektriksel gürültü nedeniyle bilgi bloğunda rastgele bir bit bozulur.
  - Tek bitlik hataya spike, birden fazla bitlik hataya burst denir. Bitin ne 0 ne 1 olduğu, bilginin tamamen silindiği durumlar da vardır.
- En çok kullanılan hata denetim metotları (ekrandan, 00:20:00)
  - FEC (Error Correction Code, Forward Error Correction, ileri yönlü hata denetimi): veri kümesine ek bitler ekler; hatayı bulur, gerekirse alıcıda düzeltmeye çalışır. Hız kaybı yaşanmaz. Çok gürültülü ortamlarda kullanılamaz. Yeniden iletimin zor ya da imkânsız olduğu bağlantılarda ve veri kümesinin küçük olduğu uygulamalarda kullanılır. Örnekler: katlamalı kodlar, BCH kodları, Hamming kodları, Reed-Solomon kodları (slaytta “Reed-Salamon”).
  - ARQ (Automatic Repeat Request, otomatik tekrar isteği): hatayı bulur ama düzeltmeye çalışmaz; alıcı, bozulan verinin yeniden gönderilmesini ister. Uygulamalarda genellikle bu kullanılır, çünkü hatayı tespit için gereken bit sayısı düzeltmek için gerekenden çok daha azdır. Örnekler: CRC kodları, seri eşlik (parity), blok eşlik, modül toplamı.
- Tekrar ederek gönderme (ekrandan, 00:24:00)
  - Kontrol biti eklenmeden gönderilen `1011010100110` gibi bir veride bir bit bozulursa alıcı bunu anlayamaz (“hata sezilemez”). Sıkıştırılmış veri açıldığında orijinal veri elde edilemez.
  - Her biti çift sayıda (bir kopya ile) yazmak: `11 00 11 11 ...`. Alınan `01` çifti hata olduğunu gösterir ama asıl bitin 0 mı 1 mi olduğu bilinemez (“hatayı sezeriz ama düzeltemeyiz”). İki bitlik bozulma (`11` yerine `00`) sezilemez. Maliyet: 1 bit yerine 2 bit.
  - Her biti iki kopya ile (üç kez) yazmak: `111 000 111 ...`. Alınan `011` grubunda hata sezilir; tek bitlik bozulmaysa çoğunluğa bakılarak `111` diye düzeltilir. İki bit bozulursa düzeltilemez. Maliyet: 1 bit yerine 3 bit.

- Her iki bit için 5 bitlik kod kelimesi (ekrandan, 00:30:00)
  - Kod tablosu: `00 → 00001`, `01 → 01010`, `10 → 10100`, `11 → 11111`. Kod kelimeleri birbirinden en az 3 bitte farklı olacak şekilde seçilmiştir.
  - Maliyet 2 bit yerine 5 bit, yani 1 bit yerine 2,5 bit. Bir bitte oluşan bozulmalar kesinlikle düzeltilebilir: alınan kelime tabloda yoksa, tek bit farkla hangi kod kelimesinden gelebileceğine bakılır.
- Hamming kodlama (ekrandan, 00:40:00)
  - 4 bitlik veriden 7 bitlik kod kelimesi elde edilen lineer bir kodlamadır (1 bit ≈ 1,75 bit). Bütün kod kelimelerinde en az 3 bitte değişim vardır; tek bitlik bozulmalar düzeltilebilir.
  - Kod kelimesi: $b_1 b_2 b_3 b_4\,(b_1 \oplus b_2 \oplus b_3)(b_1 \oplus b_3 \oplus b_4)(b_2 \oplus b_3 \oplus b_4)$. $\oplus$ parity addition (XOR) işlemidir: girişler aynıysa 0, farklıysa 1.
  - Alıcıdaki üç şart (ekrandan, 00:46:30): $P_1 = b_1 \oplus b_2 \oplus b_3 \oplus b_5$, $P_2 = b_1 \oplus b_3 \oplus b_4 \oplus b_6$, $P_3 = b_2 \oplus b_3 \oplus b_4 \oplus b_7$. Üçü de 0 ise bozulma yoktur; ilk dört bit mesajdır, son üç kontrol biti atılır.
  - Bir P_j 1 ise bozulan bit, ilgili P_j'de olup diğerlerinde olmayan bittir; tersi alınır. Örnek: $P_1 = P_2 = 1$, $P_3 = 0$ ise bozulan bit b_1'dir. Üçü de 1 ise üçünde de bulunan b_3 bozulmuştur (ekrandan, 00:50:00).
- Eşlik sınaması (parity check) (ekrandan, 00:56:30)
  - Gönderilen veride oluşan tek sayıdaki hatayı sezmek için kullanılır; düzeltme yapmaz, yeniden gönderim istenir.
  - Verideki birlerin sayısı eşlik biti ile tek ya da çift olarak düzenlenir. Tek eşlikte birlerin sayısı tekse eşlik biti 0, çift eşlikte birlerin sayısı tekse eşlik biti 1 olur.
  - İki bit bozulursa birlerin sayısının tek/çift durumu değişmez ve hata sezilemez.
- Günlük hayattan örnekler
  - UPC barkodu (Universal Product Code) (ekrandan, 01:00:00 ve 01:20:00): çizgi kalınlıkları bitlere karşılık gelir (ince siyah çizgi 1, biraz kalın 11, kalın 111, çok kalın 1111; boşluklar 0, 00, 000, 0000). Kod 1 + 5 + 5 + 1 dijittir: ürün tipi, üretici kodu, ürün kodu, kontrol dijiti. Kontrol dijiti barkodda bulunur ama rakamla yazılmaz. Sol taraftaki 7'li gruplar 0 ile başlar, 1 ile biter ve birlerin sayısı tektir; sağdakiler 1 ile başlar, 0 ile biter ve birlerin sayısı çifttir.
  - UPC kontrol dijiti: $3a_1 + a_2 + 3a_3 + a_4 + \cdots + a_{10} + 3a_{11} \equiv A \pmod{10}$, kontrol dijiti $C = a_{12} = 10 - A$.
  - ABD posta sisteminde kullanılan barkod (ekrandan, 01:25:40): her grup 2 uzun, 3 kısa çubuktan oluşur ve 0-9 arası bir dijiti gösterir; baştaki ve sondaki uzun çubuklar koruma çizgisidir. Son dijit kontrol dijitidir; bütün dijitlerin toplamı 10'a göre 0 olmalıdır. Kısa yerine uzun ya da tersi okunduğunda hata oluşur.
- CRC (Cyclic Redundancy Check) (ekrandan, 01:34:50)
  - Önce bir üreteç fonksiyonu $G(x)$ seçilir. Örnekte $G(x) = x^3 + x^2 + 1$, yani katsayılarla `1101`.
  - Üreteç fonksiyonunun derecesi CRC kodunun bit uzunluğunu verir (derece 3 ise CRC 3 bitlik); başlangıçta bu bitler 0'dır. Mesajın arkasına 3 tane 0 eklenir.
  - Mesaj, `1101` ile XOR'lanır; soldaki bit atılır. Sıradaki ilk bit 0 ise `0000`, 1 ise `1101` yazılıp yeniden XOR'lanır. 3 bit kalınca bu kalan CRC kodudur ve mesajın arkasına eklenip gönderilir.
  - Alıcı aynı işlemi CRC eklenmiş veri üzerinde yapar. Sonuç 0 ise bozulma yoktur; 0 değilse bozulma vardır ama yeri bulunamaz, verinin yeniden gönderilmesi istenir.
  - Gönderen ve alan, üreteç fonksiyonunu bilmelidir.
- Hamming ve CRC karşılaştırması
  - Hamming kodları daha kısa verilerde, CRC kodları daha uzun verilerde tercih edilir.
- MD5 (Message Digest) (ekrandan, 01:42:30)
  - Bir internet standardıdır; veri güvenliği ve bütünlüğünde kullanılır. Tam güvenli bir algoritma değildir, ama hızlı çalıştığı için özellikle internette tercih edilir.
  - İstenen her boyuttaki veriden 128 bitlik bir özet elde eder ve bu özeti 32 bitlik 4 parçaya böler. Hash fonksiyonları gibi düşünülebilir.
  - Veri 512 bitlik bloklara ayrılır; veri 512'nin katı değilse padding yapılır.
  - Tek yönlüdür, geri dönüşü yoktur. Bu yüzden kullanıcı şifreleri veritabanında MD5 özeti olarak saklanır. "Şifremi unuttum" denince sistem şifreyi söyleyemez, yeni şifre oluşturulmasını ister. Şifre e-postayla geri gönderiliyorsa şifre açık hâlde saklanıyor

demektir.
  - SHA (Secure Hashing Algorithm): SHA1, $(2^{64} - 1)$ bitlik mesajdan 160 bitlik özet üretir.
- Sezgisel sıkıştırma (intuitive compression) (ekrandan, 01:50:00)
  - Braille kodlama: görme engelliler için 6 noktalı (3x2) alfabe. $2^6 = 64$ farklı kod üretilebilir; harfler, büyük harfler ve noktalama işaretlerinden artan kodlara dilde sık geçen karakter grupları atanır (slaytta "and, for, of, the, with, ch, gh, sh, th"). Böylece 4 karakterlik "with" 6 bitle kodlanır; bu hem kodlama hem sıkıştırmadır. Müzik notaları, matematik ve programlama için de Braille türleri vardır.
  - Morse kodlama: karakterler kısa ve uzun çizgilerle gösterilir. Nokta 1 birim, çizgi 3 birim; nokta ve çizgiler arası 1 birim, karakterler arası 3 birim, kelimeler arası 6 birimdir (bkz. Detaylı Açıklamalar). 8 nokta göndermek, son kelimenin hatalı olduğunu ve silinmesini bildirir.
- Geri dönüşümsüz metin sıkıştırma (irreversible text compression)
  - Boşlukları silmek, bütün harfleri küçük ya da büyük harfe çevirmek gibi işlemlerdir. Taşınan bilgi değişmez, yalnız format değişir; ama orijinale dönülemez. Bu tür sıkıştırmalar ad hoc (veriye özel) sıkıştırmadır.
- Sparse string sıkıştırma (ekrandan, 02:00:00)
  - Sparse veri, çoğunluğu 0 olan veridir. Metinde her karakter yerine 0, her boşluk yerine 1 yazılarak L1 elde edilir ve 4'lü gruplara ayrılır.
  - L1'deki her 4'lü grup 0000 ise 0, değilse 1 yazılarak L2; aynı işlemle L2'den L3 elde edilir.
  - L1 ve L2'deki sıfır grupları silinir; L1, L2, L3 ve boşluksuz metin gönderilir. Alıcı L3'ten L2'yi, L2'den L1'i kurar ve boşlukları yerine koyar.
- Baudot kodlama (ekrandan, 02:10:00)
  - Her sembol 5 bitle kodlanır ($2^5 = 32$ kod). Aynı kod hem bir harfe hem bir rakama ya da işarete karşılık gelir (örneğin `10000` hem A hem 1).
  - LS (Letter Shift, `00001`) ve FS (Figure Shift, `00010`) kodları, sonraki kodların harf mi rakam mı okunacağını belirtir. CR, LF, ER (hata) kodları vardır; atanmamış (na) kodlar istenen özel sembole verilebilir.
- Önek sıkıştırma (front compression) (ekrandan, 02:15:00)
  - Alfabetik sıralı bir kelime listesinde her kelimenin önüne, önceki kelimeyle ortak olan soldaki karakter sayısı yazılır, ardından kalan karakterler gelir: `a`, `1ardvark`, `1back`, `3ft` (abaft), `3ndon` (abandon), `3ndoning` ...
  - Kelimeler karışık sırada olsaydı sıkıştırma olmaz, veri büyüyebilirdi.
- Run-Length Encoding (RLE) (ekrandan, 02:20:00)
  - Input stream içinde bir d verisi n defa arka arkaya geliyorsa bu bir "nd" çifti ile gösterilir; n'ye run (tekrar sayısı) denir.
  - Hem metin hem görüntü sıkıştırmada kullanılır; kayıplı ve kayıpsız versiyonları vardır. JPEG'in içinde, WinRAR ve WinZip gibi araçlarda bir aşamada RLE kullanılır.
  - Kayıplı kullanım örneği: 20, 20, 20, 20, 21, 20, 20, 20, 20 piksel değerlerinde gözün ayırt edemeyeceği 21 yerine 20 yazılırsa sıkıştırma artar ama açınca 21 geri gelmez.
  - Negatif sıkıştırma: arka arkaya tekrar eden karakterler azsa, tek karakterler için de "1x" yazıldığından çıktı girdiden büyük olur (ekrandan, 02:25:30).
  - Rakamlarla karışma sorunu: "2. all is too well" metninde `2.a2l is t2o we2l` yazılırsa çözücü "2"lerin sayı mı run mı olduğunu bilemez (ekrandan, 02:30:00). Çözüm: metinde geçmeyen özel bir escape karakteri (örnekte `@`): `2. a@2l is t@2o we@2l`.
  - RLE yalnız 3 ve daha fazla tekrarda kullanılır: 1 karakter 2 bayta çıkar, 2 karakter 2 bayt kalır, 3 karakter 2 bayta iner.
- RLE'nin temel problemleri ve MNP5 (ekrandan, 02:37:20)
  - Doğal dilde, boşluklar dışında, arka arkaya tekrar eden karakter nadirdir.
  - Tekrar sayısına 1 bayt ayrıldığı için run en çok 255'tir; daha uzun tekrarlar bölünerek yazılır.
  - Escape kodu metinde geçebilir; exe dosyalarında bütün ASCII karakterler bulunabileceği için kullanılmayan bir karakter olmayabilir. Bu durumda MNP5 (Microcom Network Protocol) yöntemi kullanılır.
  - MNP5: bir karakter 3 kez arka arkaya geliyorsa 3 kopyası yazılır, dördüncü bayta run değeri (fazlalık) yazılır: `aaa → aaa0` (3 bayt yerine 4, genişleme), `aaaa → aaa1` (4 yerine 4, sıkıştırma yok), `aaaaa → aaa2` (5 yerine 4, sıkıştırma var).
  - Performans (N: karakter sayısı, M: tekrar eden grup sayısı, L: gruplardaki ortalama

eleman sayısı) (ekrandan, 02:40:00): RLE için $[N - (M \cdot L) + 3M]/N$, MNP5 için $[N - (M \cdot L) + 4M]/N$.

Hocanın Özellikle Vurguladığı Kısımlar

- Hamming kodlama önemli bir kodlamadır
  - Hoca, ileri yönlü hata denetimi içinde en önemli yöntem olarak Hamming kodunu ayrıntılı anlattı. En çok tercih edilen yöntemdir: 1 bit yerine 1,75 bit harcanır, tek bitlik hata bulunur ve %100 doğru olarak düzeltilir.
- Yalnız tek bitlik bozulmalar düzeltilir
  - Bu derste gösterilen yöntemler tek bitlik (spike) bozulmalar içindir. Hamming koduyla 2 bitlik bozulma bazen tesadüfen düzeltilebilir, ama tek bitlik bozulma kesin düzeltilir. 2-3 bitlik bozulmalar burst hatadır; bunlarda veri yeniden istenir.
  - Veri pratikte küçük bloklar hâlinde (örneğin 8'er bit) gönderildiği için tek bitlik bozulma varsayımı yapılır.
- Sıkıştırma amacıyla kontrol biti sayısı azaltılmalıdır
  - Hata düzeltmek için veriye eklenen bit sayısı da sıkıştırmayı etkiler. Bu yüzden 1 bit yerine 3 bit kullanan tekrar yöntemine göre 2,5 bit, ondan da 1,75 bit (Hamming) kullanan yöntem daha idealdir.
- Hatayı sezmek, düzeltmekten kolay ve ucuzdur
  - Uygulamalarda genellikle yeniden gönderim istenir; hatayı tespit için gereken bit, düzeltmek için gerekenden azdır.
- RLE "yöntem deyip geçmeyin"
  - JPEG, WinRAR, WinZip gibi birçok algoritmanın bir aşamasında RLE vardır.
  - Hoca "RLE metinde mi görüntüde mi daha başarılı?" diye sordu: görüntüde daha başarılıdır, çünkü aynı renkler arka arkaya gelir; metinde arka arkaya aynı karakter, boşluklar dışında, nadirdir.
- MNP5 neden kullanılır?
  - Ağda gönderilen dosyaların büyük çoğunluğu exe ya da sıkıştırılmış dosyalardır; bunlarda özel sembol bulunamayabilir. RLE ile MNP5 arasında büyük fark olmadığı için MNP5 güvenle kullanılabilir.

Kısa Tekrar Notları

- Hata: iletimde bitlerin bozulması. Sezmek kolay, düzeltmek zor ve pahalı.
- Spike: tek bit hatası; burst: çok bitlik hata.
- FEC: ek bitlerle hata bulunur ve alıcıda düzeltilir (Hamming, BCH, Reed-Solomon). ARQ: hata bulunur, yeniden gönderim istenir (CRC, eşlik).
- Tekrar kodu: 1 kopya (2 bit) hatayı sezer, düzeltemez; 2 kopya (3 bit) tek bit hatayı düzeltir.
- 2 bit → 5 bit kod: en az 3 bit fark, 1 bit yerine 2,5 bit, tek bit hata düzeltilir.
- Hamming (7,4): $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_3 \oplus b_4$, $b_7 = b_2 \oplus b_3 \oplus b_4$; 1 bit yerine 1,75 bit.
- Hamming çözme: P_1, P_2, P_3 hepsi 0 ise hata yok; 1 olan P'lerin hepsinde olup 0 olanlarda olmayan bit ters çevrilir.
- Eşlik biti: tek sayıdaki hatayı sezer, düzeltmez; iki bit hatayı sezemez.
- UPC: $C = 10 - A$, $A = (3a_1 + a_2 + 3a_3 + \cdots + 3a_{11}) \bmod 10$.
- Posta barkodu: dijitlerin toplamı 10'a göre 0.
- CRC: $G(x)$'in derecesi kadar 0 eklenir, XOR ile bölünür, kalan CRC'dir; alıcıda kalan 0 değilse hata vardır.
- MD5: 128 bit özet, 512 bitlik bloklar, tek yönlü; SHA1: 160 bit özet.
- Braille 6 bit (64 kod), Baudot 5 bit (32 kod, LS/FS), Morse kısa/uzun çizgi.
- Sparse string: karakter 0, boşluk 1 → L1, L2, L3 + boşluksuz metin.
- Önek sıkıştırma: önceki kelimeyle ortak önek sayısı + kalan karakterler.
- RLE: "nd" çifti; 3 ve üstü tekrarda kazanç; metinde negatif sıkıştırma riski; escape karakteri; run ≤ 255.
- MNP5: 3 kopya + run baytı (`aaa2` = 5 a).

Detaylı Açıklamalar

- Hata sezme ve düzeltme yöntemlerinin maliyet karşılaştırması
  - Kontrol biti eklemeden gönderim: hata sezilmez.
  - Her bitin bir kopyası (1 bit → 2 bit): hata sezilir, düzeltilemez.
  - Her bitin iki kopyası (1 bit → 3 bit): tek bit hata düzeltilir.
  - 2 bit → 5 bit kod tablosu (1 bit → 2,5 bit): tek bit hata düzeltilir. Örnek (ekrandan, 00:30:00): mesaj `10 11 01 00 11` → `10100 11111 01010 00001 11111`. Alıcıya `00100 11111 00010 00000 11111` gelirse: `00100` tabloda yoktur, tek bit farkla `10100`'den gelir → `10`; `11111` bozulmamıştır → `11`; `00010` tek bit farkla `01010`'dan gelir → `01`; `00000` tek bit farkla `00001`'den gelir → `00`; `11111` → `11`. Sonuç `10 11 01 00 11`, gönderilenin aynısıdır.
  - Hamming (1 bit → 1,75 bit): tek bit hata düzeltilir.
- Hamming kodlama örnekleri
  - Kodlama (00:40:30): mesaj `1100`. $b_5 = 1 \oplus 1 \oplus 0 = 0$, $b_6 = 1 \oplus 0 \oplus 0 = 1$, $b_7 = 1 \oplus 0 \oplus 0 = 1$. Kod kelimesi `1100011` (slayttaki tabloyla aynı).
  - Çözme (ekrandan, 00:46:30): alınan kelime `1100100`. $P_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$, $P_2 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$, $P_3 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$. Üçü de 1 olduğundan bozulan bit üçünde de bulunan b_3'tür. b_3 ters çevrilince `1110100` elde edilir; bu tablodaki `1110` mesajının kod kelimesidir ve üç kontrol de 0 verir.
  - Derste örneklenen durumlar: yalnız $P_2 \to b_6$; $P_1, P_2 \to b_1$; üçü birden → b_3.
- UPC kontrol dijiti örneği (ekrandan, 01:20:00)
  - Ürün kodu `0 48500 00139`, kontrol dijiti barkodda yazılmayan 4.
  - $3 \cdot 0 + 4 + 3 \cdot 8 + 5 + 3 \cdot 0 + 0 + 3 \cdot 0 + 0 + 3 \cdot 1 + 3 + 3 \cdot 9 = 4 + 24 + 5 + 3 + 3 + 27 = 66$, $A = 66 \bmod 10 = 6$, $C = 10 - 6 = 4$.
  - Okunan herhangi bir dijit hatalıysa aynı işlem 4'ü vermez ve okuyucu "ürün tanınmadı" gibi bir hata verir. Üretici farklı bir polinom ya da farklı bir mod kullanabilir; okuyan program da ona göre yazılır.
- Posta barkodu örneği (ekrandan, 01:25:40)
  - Okunan sayı `2007755761`, son `1` kontrol dijitidir. Beşinci dijit okunamamış olsun: `2007?55761`.
  - $2 + 0 + 0 + 7 + ? + 5 + 5 + 7 + 6 + 1 = 33 + ?$. Toplamın 10'a tam bölünmesi için $? = 7$ olmalıdır; tek dijitlik hata böylece düzeltilebilir.
- CRC örneği (ekrandan, 01:34:50 ve 01:40:00)
  - Gönderilecek mesaj `0111101`, $G(x) = x^3 + x^2 + 1$ → `1101`. Mesajın arkasına `000` eklenir.
  - Slayttaki el yazısı işlemde kalan `111` bulunur ve `0111101111` gönderilir. Alıcı aynı işlemi yapınca `0000` elde eder: "Bozulma YOK". Üçüncü bit hatalı alınırsa (`0101101111`) sonuç `0001` olur, yani 0 değildir ve hata sezilir.
  - Ders sonrası not: Slayttaki işlemde ilk adımda mesajın ilk biti 0 olduğu hâlde `1101` ile XOR yapılmıştır. Standart CRC'de (mod 2 polinom bölmesi) baştaki bit 0 ise `0000` kullanılır; bu durumda `0111101000`'in `1101`'e bölümünden kalan `011` çıkar, gönderilen `0111101011` olur ve alıcıda kalan `000` bulunur (python ile doğrulandı). Slayttaki yol kendi içinde tutarlıdır (gönderen ve alan aynı işlemi yaptığı için), ama standart CRC değeri `011`'dir.
- MD5 ile ilgili düzeltmeler
  - Ders sonrası not: Slaytta "bu özet bilgiden orijinal veriye dönülür" yazıyor; oysa aynı slayt ve hoca MD5'in tek yönlü olduğunu, geri dönüşü olmadığını söylüyor. Doğrusu: özetten orijinal veriye dönülemez; özet, verinin değişip değişmediğini kontrol etmek için kullanılır.
  - Ders sonrası not: Slaytta MD5'in kırılabilirliğinin ispat edilmediği ve veri güvenliği için hâlâ tercih edildiği yazıyor. MD5'te çakışma (collision) üretmek büyük olasılıkla 2000'lerin ortasından beri pratikte mümkündür; MD5 bugün güvenlik ve şifre saklama için önerilmez.
- Morse aralıkları
  - Ders sonrası not: Hoca kelimeler arası boşluğu 6 birim dedi; uluslararası Morse kodunda kelimeler arası boşluk büyük olasılıkla 7 birimdir.
- Sparse string örneği (ekrandan, 02:00:00)
  - Metin: `Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__yapılıyor (_` boşluk). 64 karakter → 16 grup.
  - L1 = `0011|0000|0011|0001|1100|0010|0000|0000|0111|0010|0000|0000|1000|0110|0000|0000`

  - L2 = `1011|1100|1100|1100`, L3 = `1111` (python ile doğrulandı).
  - Gönderilen: L1 (sıfır grupları silinmiş) `0011|0011|0001|1100|0010|0111|0010|1000|0110`, L2, L3 ve boşluksuz metin.
  - Hoca, yöntemin zahmetli ve maliyetli olduğunu, ileride çok daha iyi yöntemler görüleceğini, ama veriye özel ihtiyaçlar için böyle ad hoc yöntemler geliştirildiğini söyledi. Tablo, resim ve metin içeren bir belgede her parçayı ayrı yöntemle sıkıştırmak (segmentasyon) da bu tür bir yaklaşımdır.
- Önek sıkıştırmada kazanç (02:16:48)
  - `abandoning` 10 karakter, 8 bitle 80 bit; `3ndoning` 8 karakter, 64 bit.
  - `abbey` 5 karakter, 40 bit; `3ey` 3 karakter, 24 bit.
- RLE örnekleri (ekrandan, 02:25:30)
  - `aaaaabbbbbbbbbbbccccddddddddeeeeeeeeee` → `5a12b4c9d10e`. Run değeri 12 ise "1" ve "2" ayrı ayrı değil, 12 olarak okunur.
  - Negatif sıkıştırma: `aabccdeefghijjjklmnopqrrstttuvvwwxyyyz` 38 bayttır; RLE ile 52 bayt olur.
  - Ders sonrası not: Hoca sıkıştırılmış hâli 50 bayt saydı; slayttaki sıkıştırılmış dizide `1h` eksik yazılmıştır. Tam dizi `2a1b2c1d2e1f1g1h1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z`, 52 bayttır (python ile doğrulandı). Sonuç değişmez: sıkıştırma değil genişleme olur.
- Run sınırı
  - Hoca, 257 tane a için `255a` ve ardından iki a yazılacağını anlattı.
- MNP5 örneği (02:36:57)
  - Hoca `baccdddfddd...` gibi bir dizide `cc`'den sonra run gelmediğini, `ddd`'den sonra mutlaka bir run baytı geldiğini anlattı: tek `ddd` için `ddd0`, 9 tane d için `ddd6`.
- RLE ve MNP5 performans örneği (ekrandan, 02:40:00)
  - $N = 1000$, $M = 10$, $L = 5$: RLE $= (1000 - 50 + 30)/1000 = 0{,}98$ (%2 sıkıştı); MNP5 $= (1000 - 50 + 40)/1000 = 0{,}99$ (%1 sıkıştı).
  - Özel sembol bulunabiliyorsa RLE biraz daha iyidir; ama her zaman bulunamaz ve fark küçük olduğu için MNP5 tercih edilebilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.